Ollama

Architecture analysis, an observability extension, and an identity and access management gateway with a new login page

Studied revision: `ollama/ollama` main branch, commit `42e911b` (2 October 2026). Code for part 4 is delivered separately as `ollama-iam.zip`.

Group member	Student ID	E-mail
[Name Surname]	[ID]	[e-mail]
[Name Surname]	[ID]	[e-mail]
[Name Surname]	[ID]	[e-mail]
[Name Surname]	[ID]	[e-mail]

Contents

1 Why we chose Ollama
2 Current architecture
  2.1 Problem definition and domain
  2.2 Technical problems and solutions
  2.3 Stakeholders
  2.4 Quality requirements
  2.5 Architectural views (4+1)
  2.6 Rationale
  2.7 Architectural patterns
3 Extension: an observability layer
  3.1 Choice of evolution area
  3.2 Architectural change
  3.3 Rationale
  3.4 Patterns
4 Login page and IAM module
  4.1 Requirements from Ollama
  4.2 Architecture
  4.3 The login page
  4.4 Implementation
  4.5 Verification and limits
5 Conclusion · References

1 Why we chose Ollama

Of the five candidates, Ollama is the only one whose whole reason to exist is a *resource scheduling problem on a single machine*: it has to fit multi-gigabyte neural networks into whatever GPU, Apple-silicon or CPU memory a user happens to own, and hide that difficulty behind one command (`ollama run llama3.2`). Terraform, Argo CD, Vault and Kubernetes are mature infrastructure tools whose architectures are already heavily documented; Ollama is younger, moves quickly, and its architecture is still being shaped in public. That made it a better subject for us for four reasons.

- **Visible architectural decisions.** In the revision we studied, Ollama drives llama.cpp's own `llama-server` as a subprocess and has added a second engine for Apple's MLX framework behind the same `LlamaServer` interface. Watching an interface absorb a second implementation is exactly the kind of evolution the course discusses.
- **Readable size.** The Go code base is about 435 non-test source files (~138 k lines) with clear package boundaries, small enough for a team of students to read the important paths end to end.
- **Real gaps that match the assignment.** Ollama ships with no user authentication and no built-in metrics endpoint. Both are long-standing community requests, so part 3 (extension) and part 4 (IAM) solve problems the project actually has instead of invented ones.
- **We can run it.** Every team member could install it on a laptop and observe the behaviour we describe, which the other candidates (especially Kubernetes and Argo CD) make much harder.

2 Current architecture

2.1 Problem definition and domain

Domain. Ollama belongs to *local LLM inference*: running large language models (and embedding, vision and speech models) on hardware the user controls instead of calling a hosted API. Its users want privacy (prompts never leave the machine), zero per-token cost, offline operation and freedom to choose models.

Problem. Running an open-weights model by hand means choosing a file format and quantisation, compiling an inference engine for the right GPU backend (CUDA, ROCm, Metal, Vulkan), estimating how many layers fit in video memory, writing the model's chat template, and exposing the result to applications. Ollama packages all of that: it downloads models from a registry the way Docker pulls images, works out placement on the available hardware, keeps models warm between requests, and serves them through a stable HTTP API (native, OpenAI-compatible and Anthropic-compatible) on `127.0.0.1:11434`. A `Modelfile`, deliberately modelled on a Dockerfile, lets users derive new models with their own system prompt, parameters or adapters.

2.2 Major technical problems and the project's solutions

Problem	How Ollama solves it (as found in the source)
Models are larger than the memory of many machines	Quantised GGUF weights (4–8 bit) and memory-mapped loading. The scheduler (`server/sched.go`) predicts VRAM need, picks a GPU or group of GPUs, offloads the remaining layers to CPU, and on an out-of-memory load retries with a smaller automatic context (`reduceAutoNumCtxForLoadOOM`).
Heterogeneous hardware (NVIDIA, AMD, Apple, Intel, CPU-only)	The `discover` package probes devices per platform; inference runs in bundled backend libraries selected at run time. Apple silicon additionally gets the MLX engine (`mlxrunner`).
Several models and requests compete for one GPU	A single scheduler goroutine owns all load/unload decisions; requests arrive on channels, are queued (`OLLAMA_MAX_QUEUE`), share a loaded runner (`OLLAMA_NUM_PARALLEL`) and idle models are evicted after a keep-alive period (`OLLAMA_KEEP_ALIVE`, default 5 min).
Inference engine crashes or leaks memory	Each loaded model runs in a separate subprocess (`llama-server` or the MLX runner) spoken to over local HTTP. A crash kills one model, not the server; the scheduler notices the exit and reloads.
Distributing multi-GB models	Content-addressed blobs (`sha256`) plus a JSON manifest per tag, an OCI-like layout. Shared layers are stored once; downloads are chunked, parallel and resumable (`transfer`).
Every model family has its own prompt format, tool-call and "thinking" syntax	Go templates stored as a manifest layer, plus model-specific renderers and output parsers (`model/renderers`, `model/parsers`, `harmony`, `thinking`).
Applications are written against OpenAI's or Anthropic's API	Translation middleware (`middleware/`, `openai/`, `anthropic/`) converts `/v1/chat/completions`, `/v1/responses`, `/v1/messages` into native requests and back.
Browser pages can reach a localhost API (DNS rebinding, CSRF)	Default bind to 127.0.0.1, an allowed-hosts check on loopback, and a CORS allow-list (`OLLAMA_ORIGINS`). Registry pushes are signed with a per-install ed25519 key (`auth`).

2.3 Stakeholders

Stakeholder	Expectations	Contributions
Core maintainers (Ollama, Inc.)	A simple product, a stable API, fast support for new model releases, a sustainable business (the ollama.com registry and cloud models).	Architecture, most commits, releases, review, the registry and desktop apps.
Community contributors	Their PRs reviewed; features such as metrics, auth, new backends.	Bug fixes, platform ports, issues, documentation, translations of APIs.
End users and developers	One-command install, good speed on their hardware, privacy, no cost.	Bug reports, benchmarks, word of mouth, Modelfiles shared on the registry.
Application integrators (Open WebUI, IDE assistants, agent frameworks)	API compatibility across releases, streaming, tool calling, structured output.	Most of Ollama's reach; client libraries; feedback on API design.
Upstream projects (llama.cpp / ggml, Apple MLX)	Correct attribution and upstreamed fixes.	The inference kernels Ollama depends on.

Stakeholder	Expectations	Contributions
Model publishers (Meta, Google, Alibaba, Mistral, DeepSeek, ...)	Their models run correctly with the right template on launch day.	Open weights, chat templates, sometimes direct collaboration on day-one support.
Hardware vendors (NVIDIA, AMD, Apple, Intel)	Their accelerators are detected and used well.	Drivers, SDKs, occasionally engineering help.
Operators in companies and labs	Run Ollama as a shared service: access control, monitoring, auditability, predictable resource use.	Deployment recipes (Docker, Kubernetes), feature requests such as #3144 (metrics).
Security researchers	Responsible handling of reports.	Findings such as exposed unauthenticated servers and malformed-GGUF memory bugs.

2.4 Quality requirements

We ranked the quality attributes by how strongly the code shows the maintainers optimising for them, and wrote one concrete scenario for each (source → stimulus → response measure), as in the quality-attribute-scenario technique from the lectures.

Attribute	Scenario	Our reasoning
Usability	A new user types `ollama run llama3.2` on a fresh laptop → the model downloads, loads and answers with no configuration.	The top priority: every default (bind address, context length chosen from VRAM, keep-alive, automatic GPU layers) exists so users never need to tune.
Performance	A warm model receives a chat request → first token in well under a second; generation speed close to raw llama.cpp.	Keep-alive, runner reuse and a thin Go layer over native kernels keep overhead small. Cold loads are the main cost, which is why eviction is lazy.
Resource efficiency	A 14 B model is requested on a 12 GB GPU → it loads with partial CPU offload instead of failing.	Memory estimation, OOM retry with smaller context, quantised KV cache (`OLLAMA_KV_CACHE_TYPE`).
Portability	The same release runs on macOS, Windows, Linux, Docker, on CUDA, ROCm, Metal, Vulkan or CPU.	Runtime device discovery and per-backend libraries; Go makes the server itself trivially cross-compiled.
Interoperability	An app written for the OpenAI SDK changes only its base URL → it works.	Translation middleware is cheaper than asking the ecosystem to adopt a new API.
Modifiability	A new model architecture is released → support ships within days.	Engine behind the `LlamaServer` interface; templates/parsers per model family; upstream llama.cpp tracked closely.
Availability / fault isolation	A runner segfaults on a malformed input → only that request fails; the server keeps serving.	Subprocess per model; the scheduler detects exit and frees the slot.
Security	A web page in the user's browser tries to call the local API → blocked by host/origin checks.	Secure *for a single local user*. As soon as the port is exposed there is no authentication, which is exactly part 4's subject.
Scalability	Many users share one server → requests queue; scaling out needs an external load balancer.	Deliberately out of scope: Ollama is single-node by design.

Attribute	Scenario	Our reasoning
Observability	An operator wants request rate, queue depth and GPU memory over time → only logs and `/api/ps` exist.	Weakest attribute today; the motivation for part 3.

> The ranking reveals the architecture's central trade-off: Ollama spends its complexity budget on **usability, performance and portability for one user**, and accepts weak **security, scalability and observability for shared use**. Our extension and IAM module both target the shared-use side without hurting the single-user side (both are opt-in).

2.5 Architectural views

We follow Kruchten's 4+1 view model: scenarios (use cases) tie together a logical view (key abstractions), a development view (code organisation), a process view (run-time interaction and concurrency) and a physical view (deployment). All diagrams were drawn by us in PlantUML from reading the source code; Ollama does not publish official architecture diagrams.

Scenario view

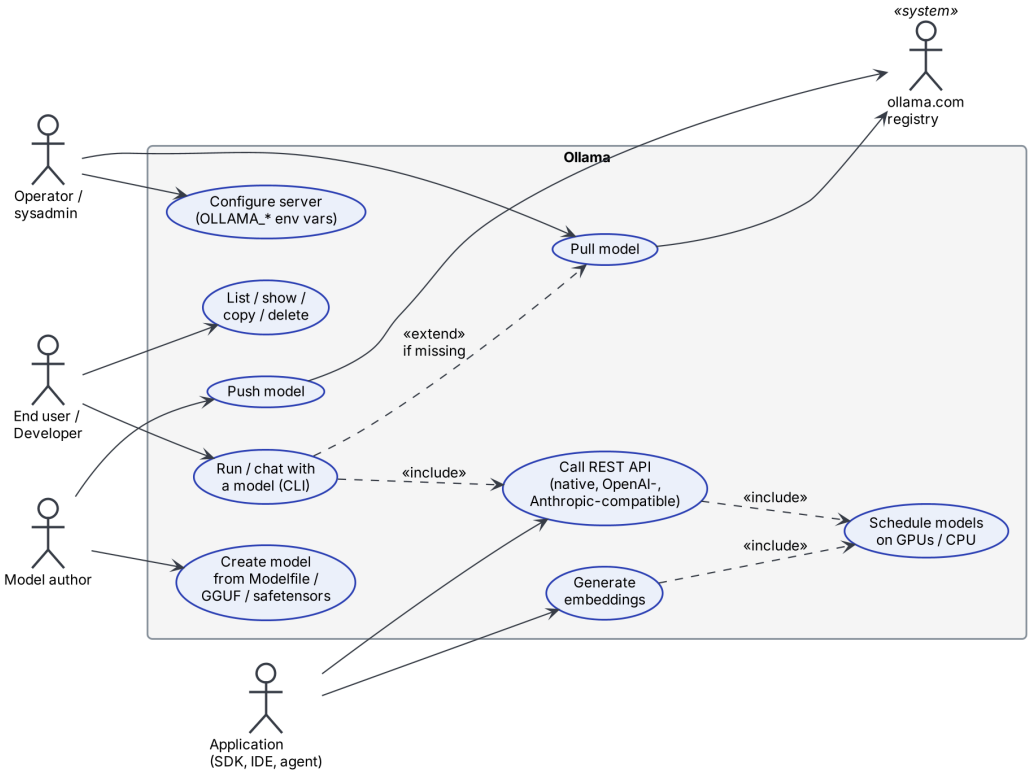


Figure 1. Use cases. Running a model includes calling the API and scheduling; pulling extends it when the model is missing.

Development view

The repository is organised by responsibility, not by layer names, but four groups emerge: clients (`cmd`, `app`, and the Go client in `api`), the HTTP layer (`server` plus compatibility middleware), model handling (registry transfer, Modelfile parsing, file-format readers, templates and parsers), and execution (scheduler, runner wrappers, device discovery and the vendored llama.cpp). Dependencies point downward; only `envconfig` is read everywhere.

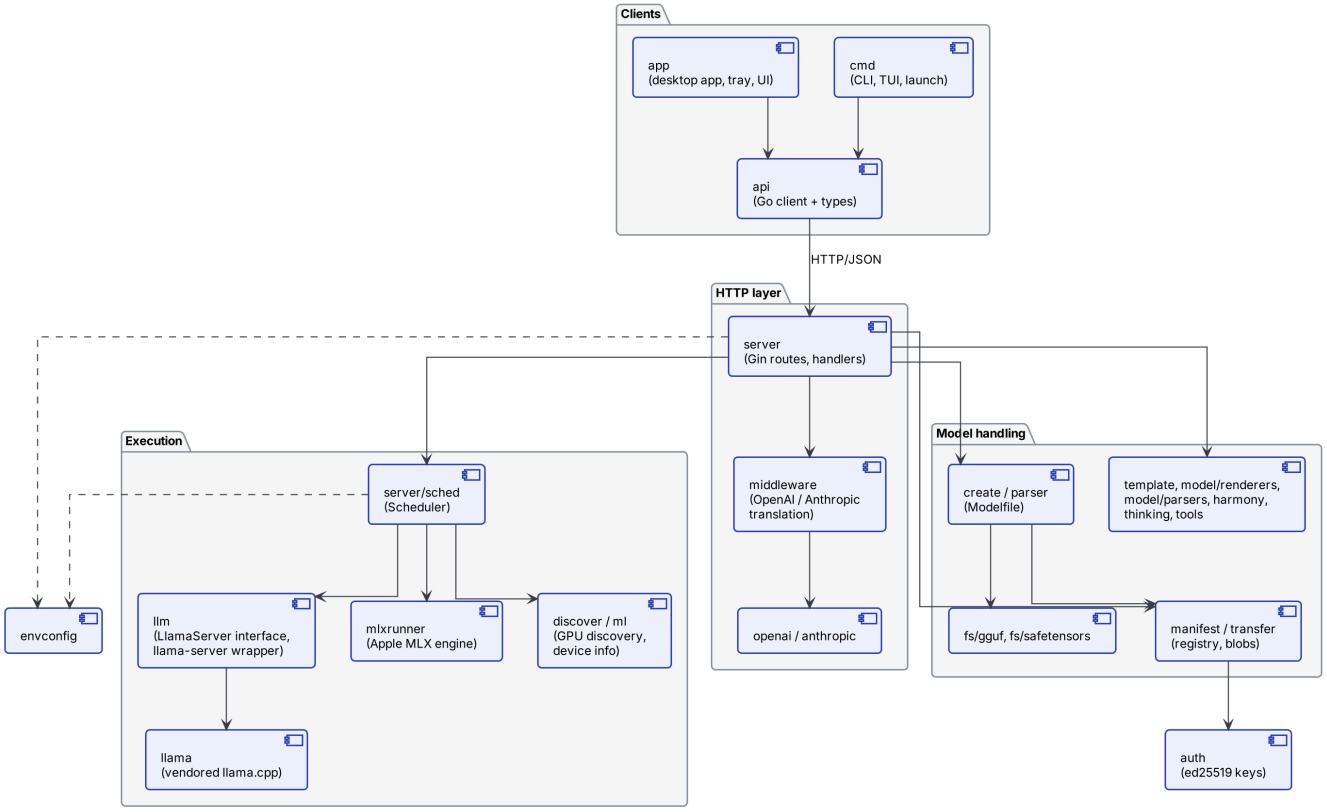


Figure 2. Package (development) view of the Go code base.

Logical view

The core abstraction is the split between **what** to run (`Model`, built from a `Manifest` of content-addressed `Layer`s) and **where** it runs (`runnerRef` wrapping a `LlamaServer`). The `Scheduler` is the only class that creates or destroys runners; handlers only borrow them, which the reference count enforces. `LlamaServer` is an interface with two implementations, so engines are interchangeable.

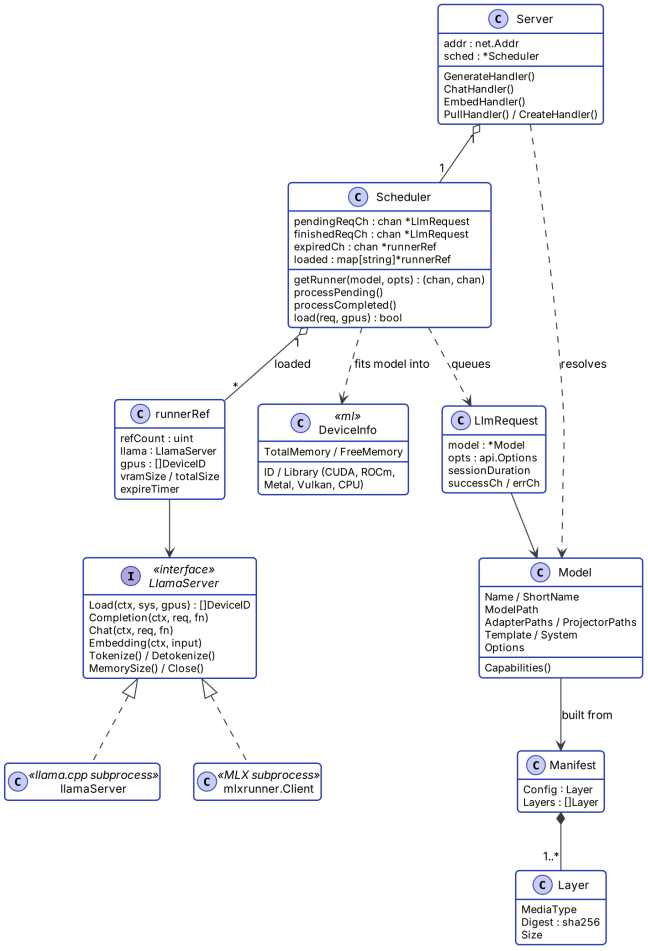


Figure 3. Logical view: key classes (Go structs and interfaces) and their relations.

Physical view

Everything runs on the user's machine. The server is one Go process; each loaded model is a child process that maps the weights from the local store and talks to the accelerator. The only network dependencies are the registry (pull/push, signed with the install's ed25519 key) and optional cloud models, which `OLLAMA_NO_CLOUD` switches off.

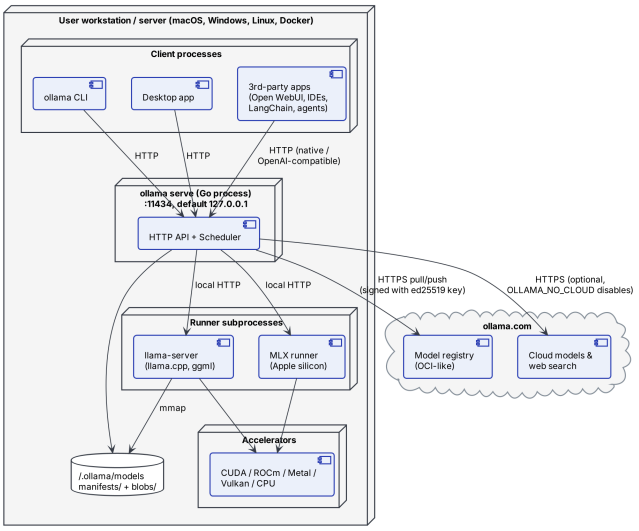


Figure 4. Deployment (physical) view.

Process view

Figure 5 follows one chat request. Concurrency is the interesting part: HTTP handlers run on many goroutines, but every load and unload decision is serialised through the scheduler's channels (`pendingReqCh`, `finishedReqCh`, `expiredCh`). This avoids locks around GPU memory accounting, at the cost of the scheduler being a single point through which all model placement flows.

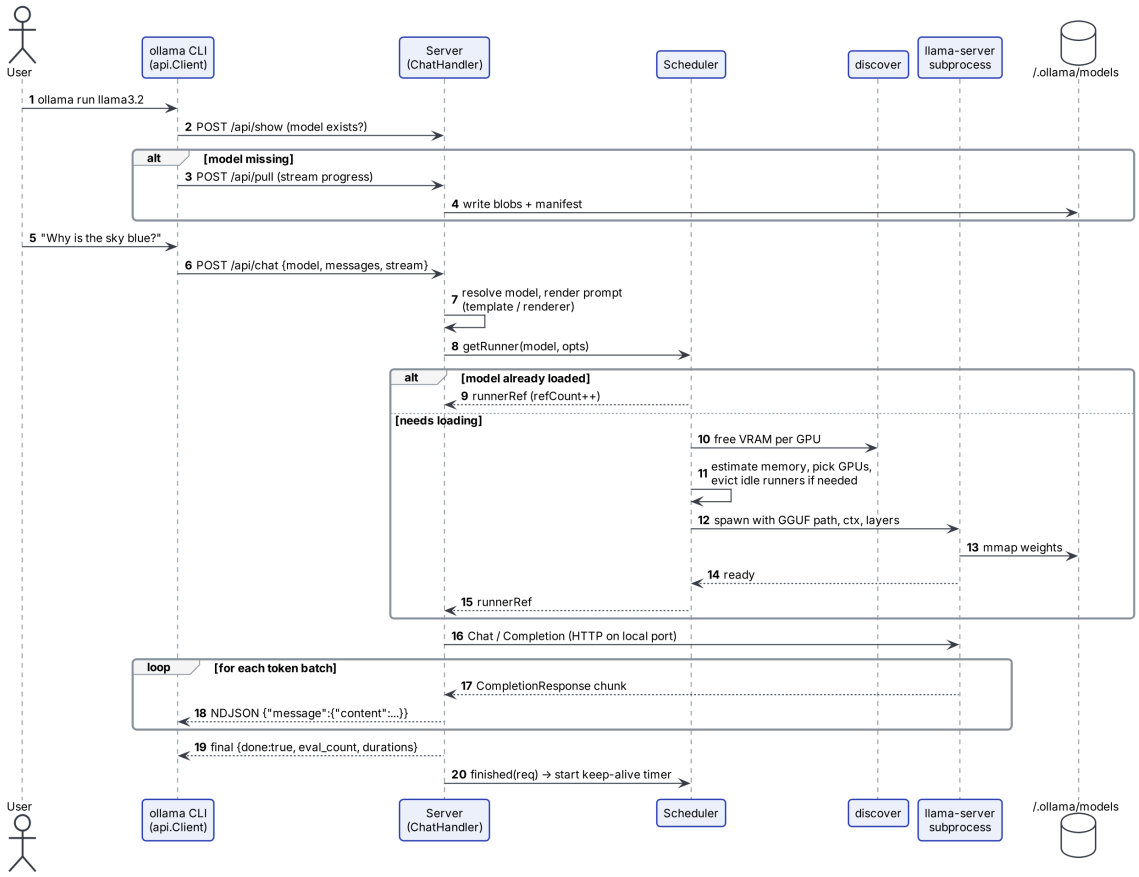


Figure 5. Process view: `ollama run` with a cold start, then token streaming.

2.6 Rationale behind the architecture

- **Client–server even on one machine.** Putting the model behind a long-lived local server lets many tools (CLI, desktop app, editors, agents) share one loaded copy of the weights. A library would load the model once per process.
- **Go for the control plane, C/C++ for the data plane.** Go gives easy cross-compilation, goroutines and a small static binary; the arithmetic stays in llama.cpp and MLX, which are where the GPU expertise lives.
- **Out-of-process runners.** Keeping the engine in a subprocess trades a little latency (local HTTP) for fault isolation, independent upgrades of llama.cpp, and the ability to add MLX without touching the scheduler.
- **Docker-like model distribution.** Reusing the mental model developers already have (pull, tag, push, a build file) lowers the learning cost, and content addressing gives deduplication and integrity for free.
- **Centralised scheduling.** GPU memory is the scarcest resource; one owner of all placement decisions makes them consistent and testable (the scheduler's dependencies are injected as functions such as `loadFn` and `getGpuFn`, so tests can fake hardware).
- **Compatibility as an adapter, not a rewrite.** The native API stays simple; OpenAI/Anthropic shapes are translated at the edge.

2.7 Architectural patterns

Pattern	Where it appears
Client–server	CLI, desktop app and third-party apps are clients of `ollama serve` over HTTP.
Layered architecture	HTTP layer → model handling → scheduling → runner → backend library; each layer only calls the one below.
Master–worker (process isolation)	The server process supervises one worker subprocess per loaded model.

Pattern	Where it appears
Producer–consumer / message queue	Handlers produce `LlmRequest`s on channels; the scheduler goroutine consumes them.
Strategy (via interface)	`LlamaServer` with llama.cpp and MLX implementations chosen per model.
Adapter	OpenAI- and Anthropic-compatible middleware wraps the native handlers.
Pipes and filters	Request path: render template → tokenize → generate → parse thinking / tool calls → stream.
Repository + content-addressable storage	Manifests and sha256 blobs in `~/.ollama/models`, mirrored by the remote registry.

3 Extension: an observability layer

3.1 Choice of evolution area and reasoning

Section 2.4 shows observability as Ollama's weakest quality attribute. Today an operator has three tools: server logs, `GET /api/ps` (which models are loaded right now) and the timing fields returned at the end of each response. None of them answers the questions operators actually ask: *How long do requests wait in the queue? How often do models reload because of eviction or OOM? Which model and which client consume the GPU? Is p95 latency getting worse after an upgrade?*

The demand is concrete. The GitHub request for a Prometheus `/metrics` endpoint (issue #3144) is still open with more than a hundred comments, a separate request asks for OpenTelemetry tracing (#9254), and at the time of writing two competing pull requests propose different minimal implementations (#16998 and #18508). Commenters report keeping private forks just to get metrics, and third-party exporters exist that poll the API from outside. We chose this area because (a) it is the most requested missing capability for shared deployments, (b) the maintainers' hesitation is *architectural* (dependency weight, where to hook in, what to expose) rather than about whether it is useful, and (c) it complements part 4: once there are many users behind an IAM gateway, per-user and per-model usage becomes essential.

> **Design goals.** G1 zero cost when disabled; G2 never slow down or block the inference path; G3 no new mandatory dependencies; G4 standard formats (Prometheus text, OTLP) so existing tooling works; G5 one instrumentation point per event, many consumers.

3.2 Architectural change

We add a `telemetry` package inside the server. Existing components do not call exporters directly; they *emit typed events* at five points they already know about: request start, queue wait, model load, token generation and request end. A bounded, non-blocking event bus fans each event out to independent observers: a metrics registry, a trace builder and a usage ledger. Exporters then present that state in standard forms: Prometheus text on `GET /metrics` (pull), OTLP spans to a collector (push), and a JSON usage summary on `GET /api/usage` that the desktop app shows in a new "Activity" tab.

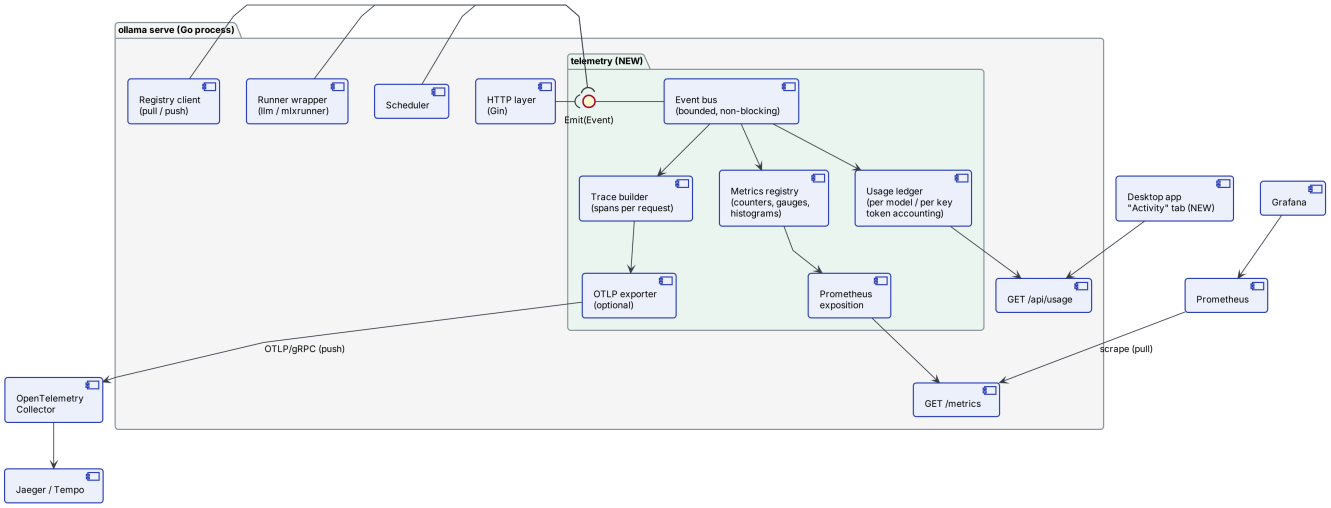


Figure 6. Component view of the extension (new parts shaded green).

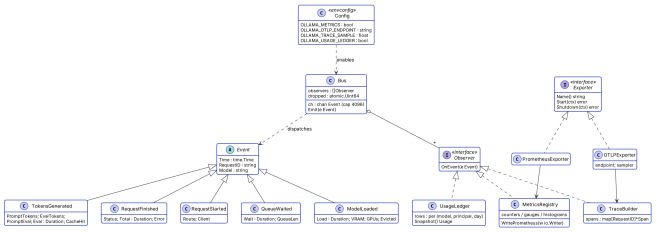


Figure 7. Logical view of the telemetry package.

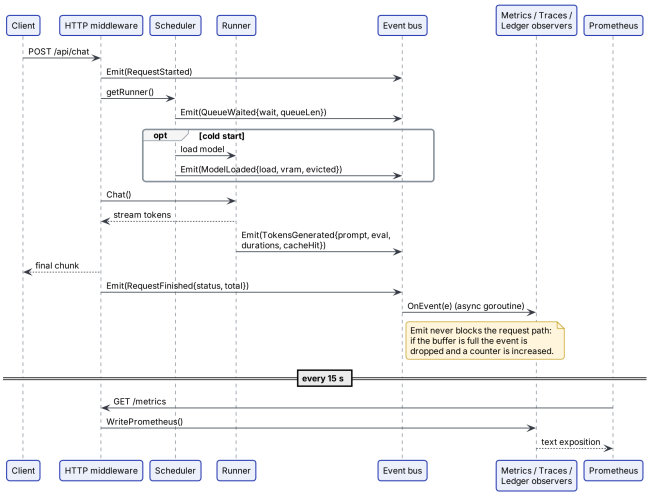


Figure 8. Process view: events emitted during one request, and a Prometheus scrape.

Metric catalogue. Names follow Prometheus conventions; labels are kept low-cardinality (model and route only, never prompt content or user identifiers on `/metrics`).

Metric	Type	Answers
`ollama_requests_total{route,model,status}`	counter	Traffic and error rate per model.
`ollama_request_duration_seconds{route,model}`	histogram	p50/p95/p99 latency.
`ollama_queue_wait_seconds`, `ollama_queue_depth`	histogram, gauge	Is the server saturated? (vs. `OLLAMA_MAX_QUEUE`)
`ollama_model_load_seconds{model,reason}`	histogram	Cold-start cost; `reason` = first, evicted, oom_retry.
`ollama_tokens_total{model,kind}`	counter	Prompt vs. generated tokens; throughput (tokens/s).
`ollama_prompt_cache_hits_total{model}`	counter	KV-cache reuse effectiveness.
`ollama_loaded_models`, `ollama_model_vram_bytes{model,gpu}`	gauge	Memory pressure per GPU.
`ollama_telemetry_events_dropped_total`	counter	Honesty about G2: how much telemetry was shed.

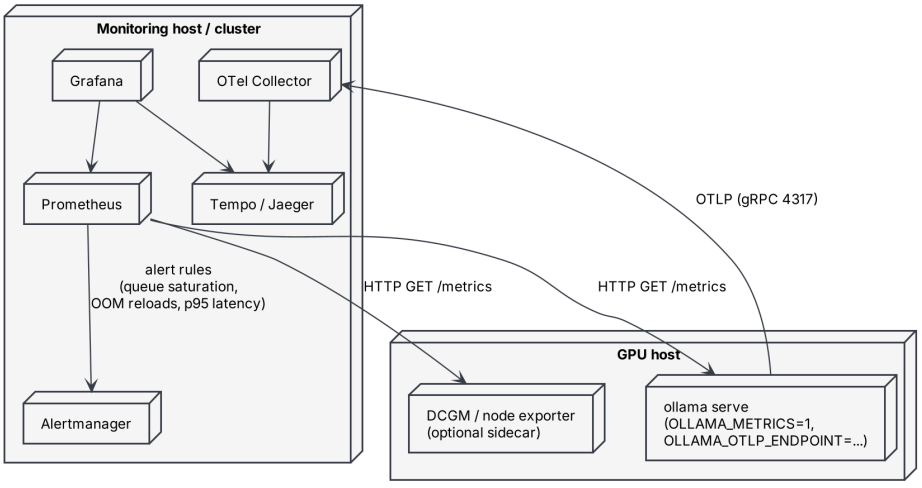


Figure 9. Physical view: a typical monitoring deployment around one Ollama node.

3.3 Rationale behind the new architecture

- **Events instead of direct metric calls.** If the scheduler incremented Prometheus counters directly, every future consumer (tracing, usage, a UI) would add another call at the same line. With events the scheduler knows nothing about consumers (G5); adding the usage ledger needed no change in the scheduler.
- **Non-blocking emit.** `Emit` does a `select` with a `default` branch: if the 4096-slot buffer is full, the event is dropped and counted. Inference latency can never be hurt by a slow exporter (G2). Dropped events are themselves a metric, so the trade-off is visible.
- **Off by default, standard formats.** `OLLAMA_METRICS=1` registers the observers and the endpoint; without it, `Emit` is a nil-check (G1). Prometheus text is written by hand (as PR #16998 also argues) so no client library is required; OTLP is a build-tagged optional exporter (G3, G4).
- **Pull for metrics, push for traces.** Pull fits Ollama's local nature (nothing leaves the machine unless something scrapes it); traces are high-volume and sampled (`OLLAMA_TRACE_SAMPLE`), so push to a collector is the usual design.
- **Usage ledger separate from metrics.** Per-client accounting has high cardinality and privacy implications, so it is kept out of `/metrics` and exposed only through an authenticated JSON endpoint (or, in a gateway deployment like part 4, keyed by the gateway's `X-Forwarded-User`).

Trade-offs accepted. Under extreme load some telemetry is lost; histograms cost a little memory per model; the new endpoints enlarge the attack surface, so `/metrics` obeys the same host checks as the rest of the API.

3.4 Architectural patterns in the new architecture

Pattern	Role in the extension
Publish–subscribe / Observer	Server components publish events; metrics, traces and ledger subscribe through the `Observer` interface.
Event bus (implicit invocation)	A bounded channel decouples emitters from consumers in time; consumers run on their own goroutine.
Strategy / plug-in	`Exporter` interface with Prometheus and OTLP implementations selected by configuration.
Sidecar	Hardware-level GPU metrics stay in a separate exporter process (e.g. DCGM), not inside Ollama.
Load shedding	Drop-and-count when the bus is full protects the critical path.

4 Login page and IAM module

4.1 Requirements derived from Ollama

An IAM design only makes sense against the system it protects. Reading `server/routes.go` gave us Ollama's complete API surface and the following facts that shape the requirements: the API has **no authentication**; anyone who can reach the port can not only run models but also **pull** (disk and bandwidth), **create**, **copy** and **delete** them; and responses are **streamed** as NDJSON or server-sent events. Exposed instances are a documented problem: Cisco's 2025 Shodan study found more than a thousand reachable servers, many answering prompts without any authentication.

ID	Requirement	Comes from
R1	Ollama itself must not be modified; the protection sits in front of it.	Ollama releases often; a fork would fall behind.
R2	Separate permissions for reading, inference, pulling, pushing and model management.	The route groups in `routes.go` have very different costs and risks.
R3	Per-user model allowlists (glob patterns, e.g. `llama3.2*`).	A 70 B model occupies a whole GPU; some models have restrictive licences.
R4	People can create their own account, but only for an e-mail address they control; they then sign in with a password and an optional authenticator-app code.	A shared lab server should not need an admin for every new student.
R5	A new account must not get inference rights by default unless it comes from a trusted organisation.	Inference is the scarce, costly resource (§2.4).
R6	Streaming responses pass through unbuffered; errors keep Ollama's `{"error": "..."}` shape.	Token streaming is core to Ollama's UX; clients already show these messages.
R7	Every security decision is audited.	Shared lab servers need accountability.
R8	Zero-configuration start like Ollama (one command, SQLite, sensible defaults).	Ollama's usability priority (§2.4).

Roles. Four roles map onto permissions that mirror the route groups: **Viewer** (`system:read`, `model:list`), **Developer** (+ `model:run`), **Model operator** (+ `model:pull`, `model:manage`, `audit:read`) and **Administrator** (everything, including `model:push` and `iam:admin`). **Self sign-up** is open to anyone (R4); the e-mail domain sets the starting role (R5): addresses at a configured trusted domain (default `bilkent.edu.tr`, including subdomains such as `ug.bilkent.edu.tr`) start as Developer, all others as Viewer until an administrator upgrades them.

4.2 Architecture

The IAM module is a **gateway** placed in front of Ollama, which is left bound to `127.0.0.1` so the gateway is the only way in. Inside the gateway, authorization follows the XACML split: a **Policy Enforcement Point** (the reverse proxy) intercepts every call, asks a **Policy Decision Point** for a decision, and forwards or rejects. Identities, credentials and sessions live in an `IAMService`; a separate `SignupService` owns pending registrations and sends confirmation codes through a `Mailer` interface (SMTP in production, a fake in tests); everything is written to an append-only audit log.

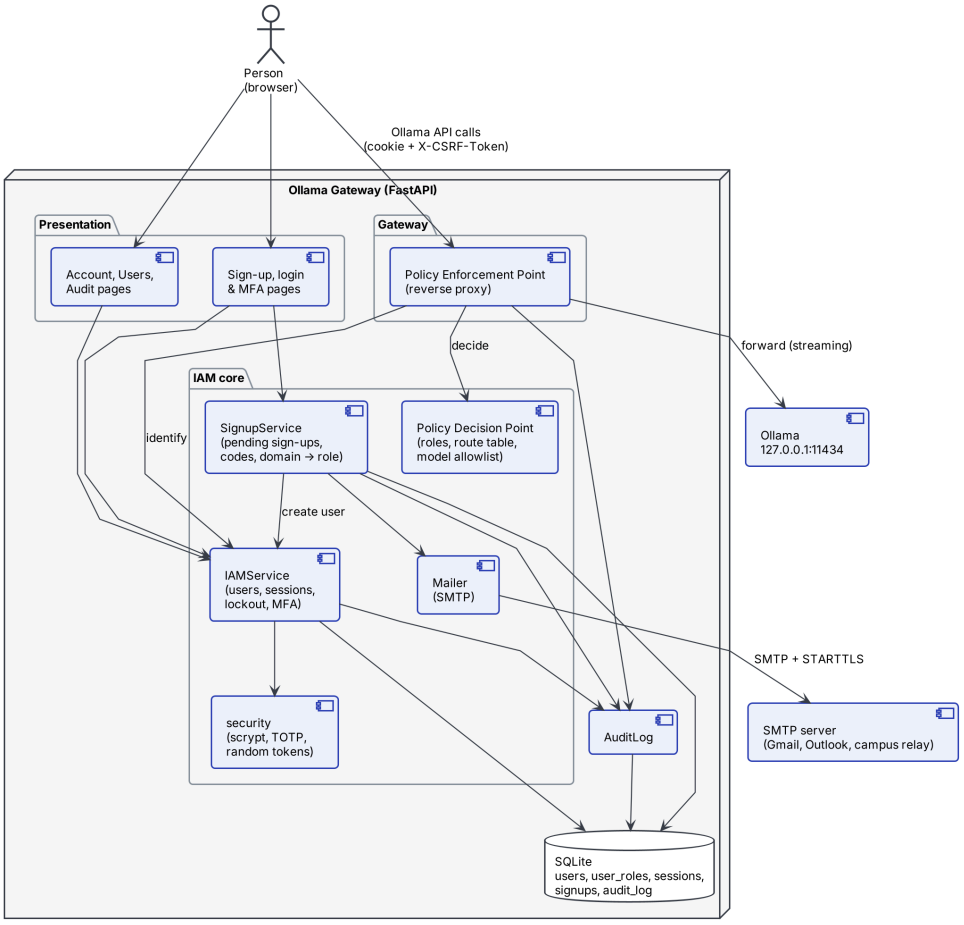


Figure 10. Component view of the gateway and IAM module.

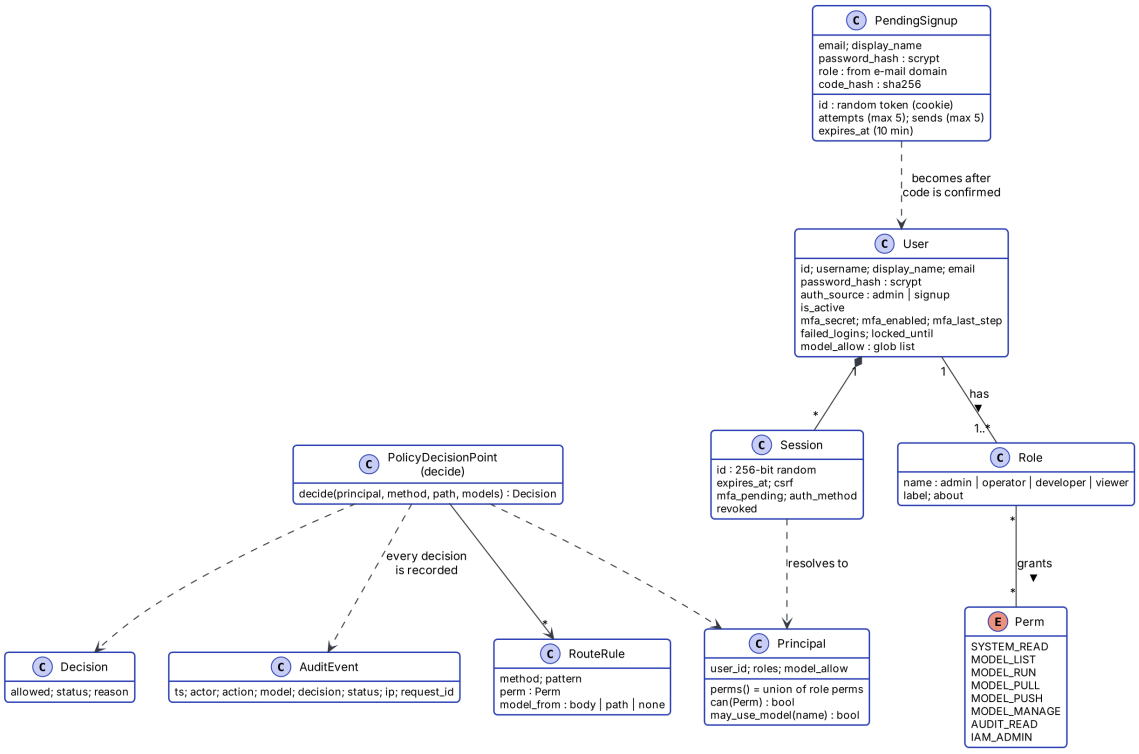


Figure 11. Logical view: the IAM domain model. A pending sign-up becomes a user only after its code is confirmed.

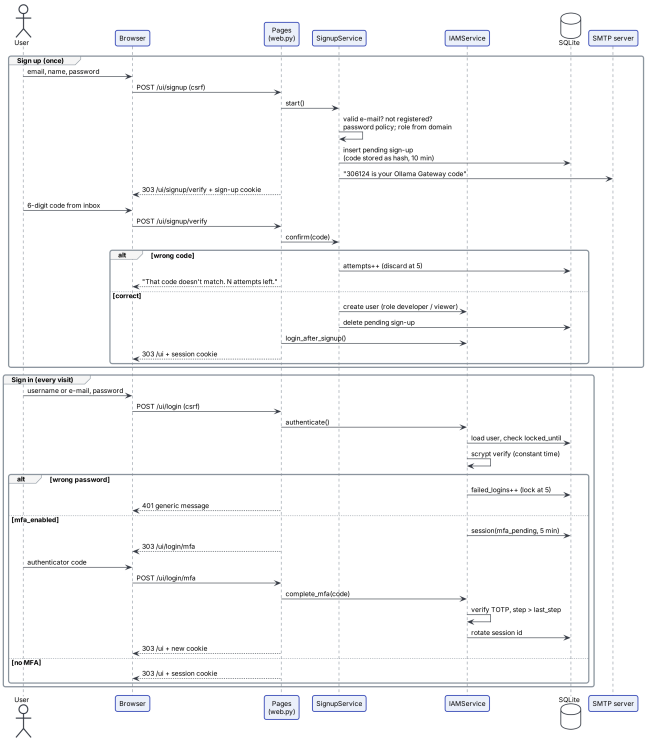


Figure 12. Process view: sign-up with an e-mailed code, then sign-in with optional second factor.

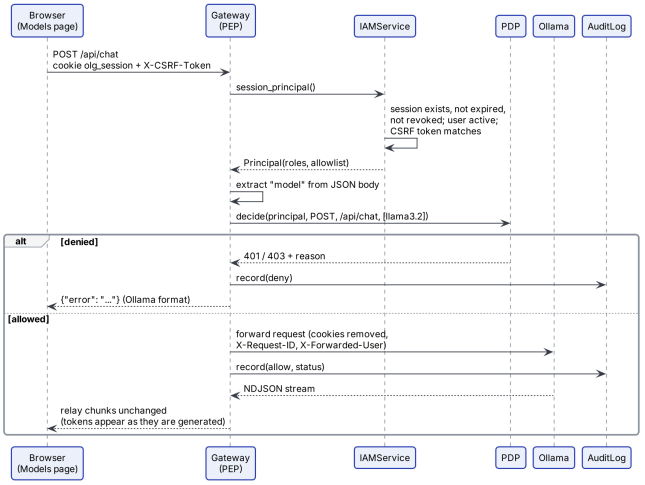


Figure 13. Process view: an API call through the enforcement point.

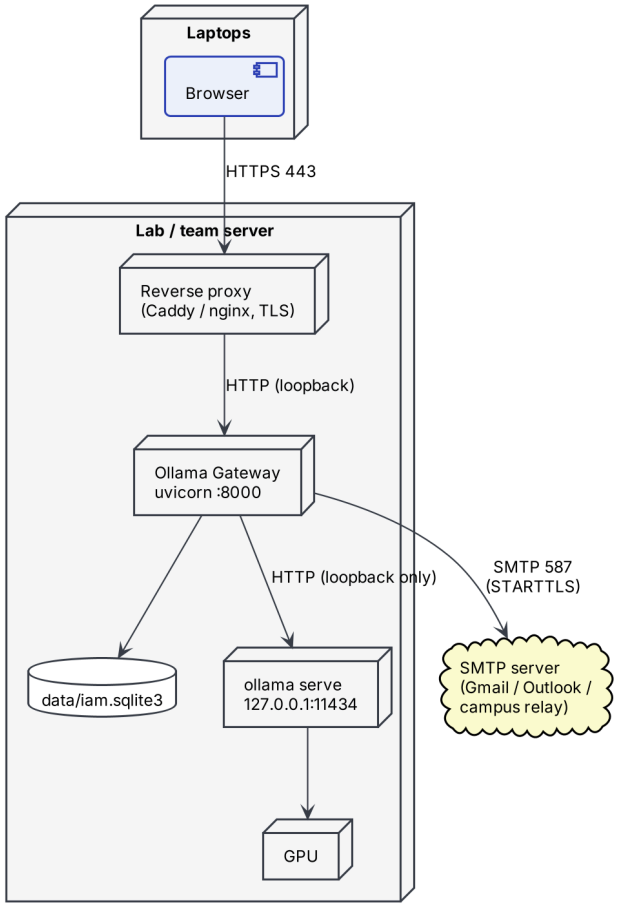


Figure 14. Physical view: recommended deployment behind TLS, with an SMTP server for codes.

Rationale and patterns

Decision / pattern	Why
Gateway (reverse proxy) instead of patching Ollama	Satisfies R1: works with every Ollama release, can be adopted or removed without touching the server, and matches the Ollama FAQ, which already describes exposing the server through a proxy such as Nginx.

Decision / pattern	Why
PEP / PDP separation	All access logic is one pure function, `decide()`, that is trivially unit-tested (Figure 13); the proxy only enforces.
Role-based access control + model allowlist (attribute)	Roles cover the route groups (R2); the allowlist is an attribute check on the requested model (R3) — a small RBAC+ABAC hybrid.
Default deny route table	Every Ollama route is listed with its permission; anything new or unknown returns 404 until it is classified.
Layered internals	Presentation (pages) → application services (`IAMService`, `SignupService`) → repository (`Database`); the proxy and pages never touch SQL.
Pipes and filters request pipeline	identify → extract model → decide → forward → filter listing → audit (Figure 13).
Proof of e-mail before an account exists	Pending sign-ups live in their own table; nothing is added to `users` until the code is confirmed, so typos and fake addresses never create accounts.
Dependency inversion for e-mail	`SignupService` depends on a `Mailer` interface, not on SMTP; tests inject a fake and any provider (Gmail, Outlook, a campus relay) is just configuration.
Least privilege by default	Untrusted domains start as Viewer (R5); an admin, not the sign-up form, grants inference to outsiders.
Server-side sessions	Unlike stateless JWT sessions, they can be revoked instantly (on disable, on logout).

4.3 The login page

Ollama's own sign-in (in the desktop app) only connects a local install to an ollama.com account; there is no page that protects a server. Our page is designed for the person who is about to use *a specific inference server*, so beside the form it shows that server's real state: a three-node route (You → Gateway → Ollama) whose links turn green as the health check succeeds, with the measured round-trip to Ollama. If Ollama is down the second link turns red before the user even signs in, which turns the most common support question ("is it me or the server?") into something visible. The health endpoint deliberately reveals no version or model names to anonymous visitors.

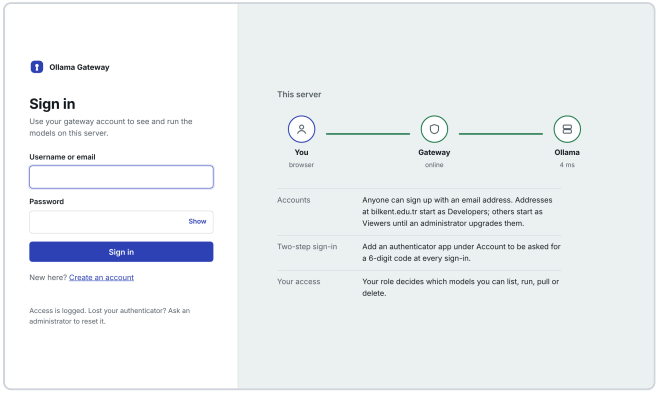


Figure 15. Sign-in page (desktop, light theme).

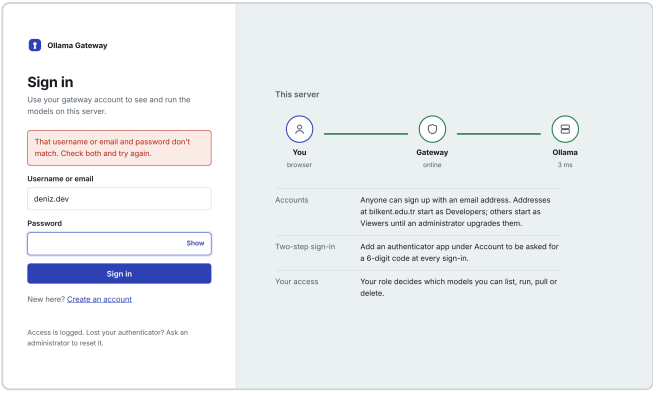


Figure 16. Failed attempt: one generic message, username kept.

Usability details: users can sign in with a username or their e-mail; a show/hide password toggle and a Caps Lock warning; the submit button disables itself to prevent double posts; error text says what to do next; the same message is used whether the account or the password is wrong. After five failures the account locks for 15 minutes with a message that says so. The second-factor screen uses six digit boxes that accept paste and submit automatically.

Creating an account

"Create an account" asks for an e-mail, a name and a password. While the e-mail is typed, a hint tells the person which role they will start with. Submitting stores a *pending* sign-up and e-mails a 6-digit code; the account exists only after the code is entered on the next page, and the person is then signed in directly. Codes expire after 10 minutes, five wrong codes discard the attempt, and "send a new code" works at most once a minute. If the server has no SMTP settings, the page says sign-up is off and an administrator can still add users.

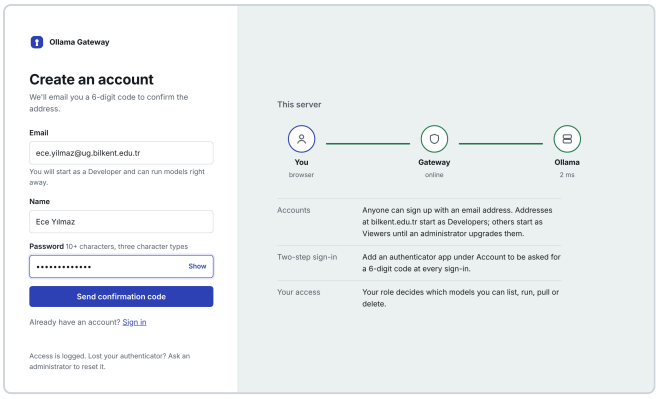


Figure 17. Sign-up form; the hint shows the starting role for a Bilkent address.

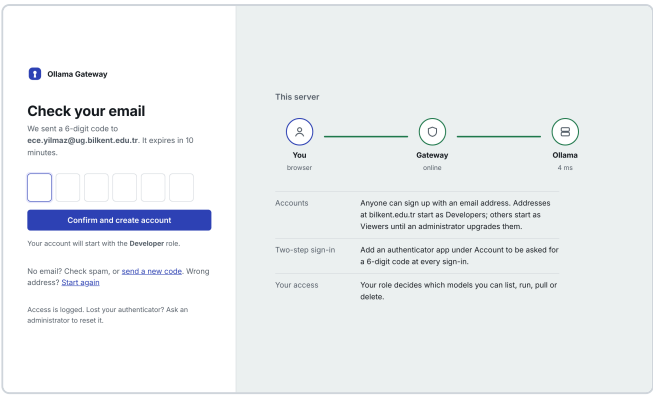


Figure 18. Code entry, with resend and start-again links.

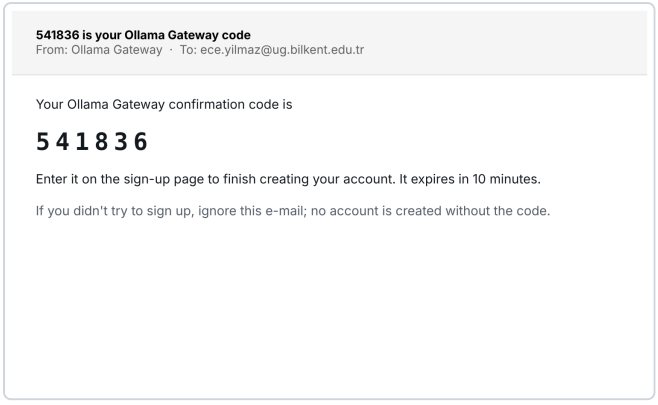


Figure 19. The confirmation e-mail as received over SMTP.

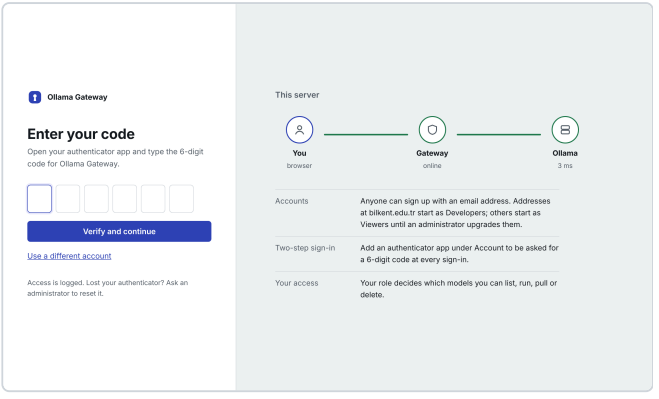


Figure 20. Optional second factor (TOTP) at sign-in.

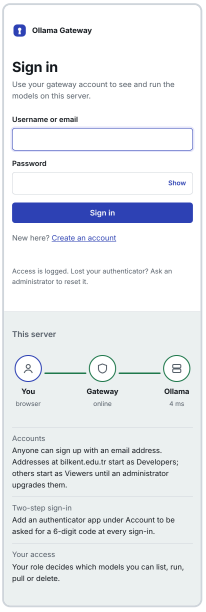


Figure 21. Phone layout.

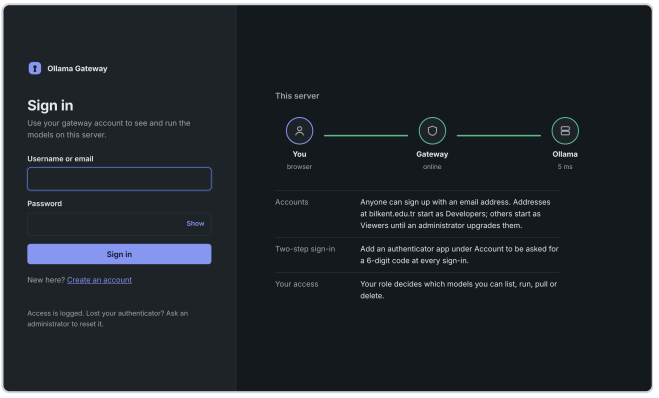


Figure 22. Dark theme follows the OS setting.

After sign-in

The **Models** page lists only the models the user may use, shows their effective permissions, and has a small prompt box that calls Ollama *through* the gateway, so a 403 for a model outside the allowlist is shown in place (Figure 23). A viewer sees the list but no prompt box. **Account** handles authenticator enrolment by QR code and password change. Administrators manage users, roles and model access, and read the audit log.

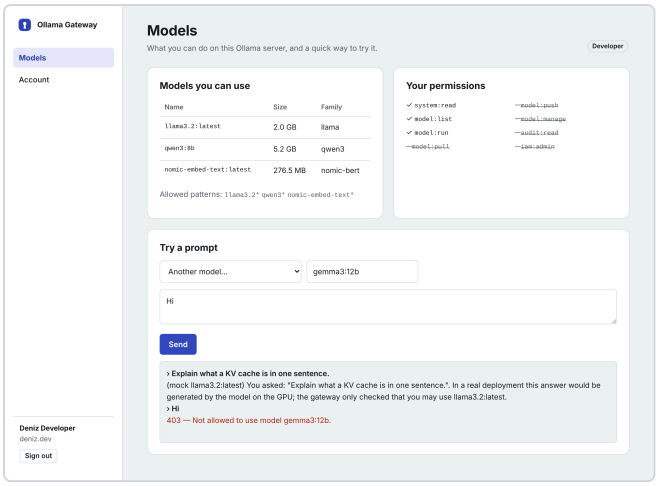


Figure 23. Developer: allowed models, permissions, and a denied request to `gemma3:12b`.

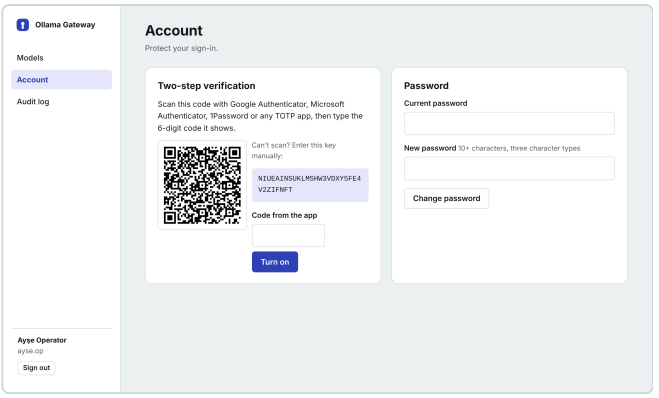


Figure 24. Enrolling an authenticator app.

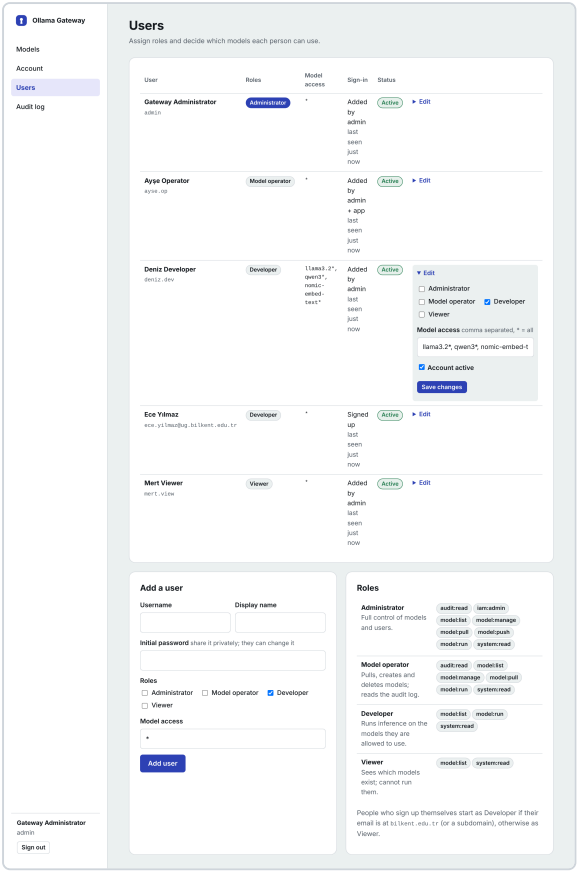


Figure 25. Administrator: users (including the self-registered one), roles and model access.

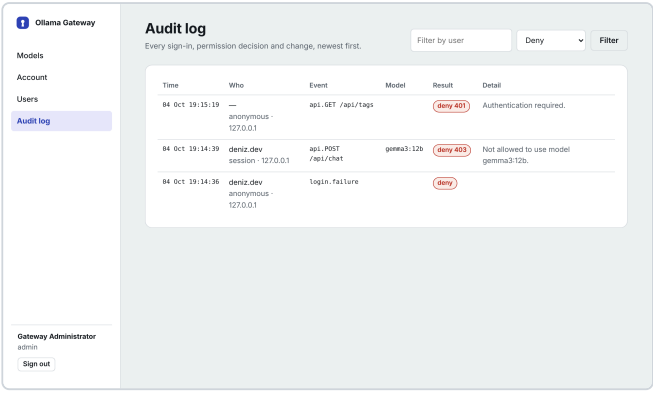


Figure 26. Audit log filtered to denials: a model outside a developer's allowlist (403), an anonymous API call (401) and a failed sign-in.

4.4 Implementation

Python 3 with FastAPI, about 2,000 lines of Python including tests. Cryptography uses the standard library so that every algorithm is visible: scrypt password hashing ($N = 2^{14}$, $r = 8$, $p = 1$, per-user salt), RFC 6238

TOTP written from scratch, SHA-256 of confirmation codes, and `smtplib` with STARTTLS for e-mail. Persistence is a single SQLite file (users, user_roles, sessions, signups, audit_log). The module layout follows Figure 10 one-to-one (`policy.py` = PDP, `proxy.py` = PEP, `iam.py` = IAMService, `signup.py` = SignupService, `mailer.py` = Mailer, `web.py` = presentation).

The route table is the heart of the design; it mirrors Ollama's router line by line:

```
ROUTE_RULES = [
    _r("GET",    "/api/tags",            Perm.MODEL_LIST, "none"),
    _r("POST",   "/api/chat",            Perm.MODEL_RUN),         # model taken from JSON body
    _r("POST",   "/v1/chat/completions", Perm.MODEL_RUN),
    _r("POST",   "/api/pull",            Perm.MODEL_PULL),
    _r("DELETE", "/api/delete",          Perm.MODEL_MANAGE),
    _r("GET",    "/v1/models/:model",    Perm.MODEL_LIST, "path"),  # model taken from URL
    ...                                                              # 25 rules in total
]

def decide(principal, method, path, models) -> Decision:
    matched = match_route(method, path)
    if matched is None:          return Decision(False, 404, "Route is not exposed through the
gateway.")
    if principal is None:        return Decision(False, 401, "Authentication required.")
    if not principal.can(rule.perm):
                                 return Decision(False, 403, f"Missing permission
{rule.perm.value}.")
    blocked = [m for m in models if not principal.may_use_model(m)]
    if blocked:                  return Decision(False, 403, f"Not allowed to use model
{blocked[0]}.")
    return Decision(True, 200, "ok")
```

The starting role of a sign-up is one small rule: `developer` if the address's domain equals a trusted domain or ends with `"." + domain`, otherwise `viewer`. Model names are normalised before matching (`llama3.2` → `llama3.2:latest`, registry prefix removed) so an allowlist cannot be bypassed by spelling a model differently. Listing endpoints (`/api/tags`, `/v1/models`) are filtered so users do not even see models they cannot use. The proxy forwards with `httpx` in streaming mode and relays raw chunks, so NDJSON and SSE streams reach the client token by token (R6); it strips the session cookie and adds `X-Request-ID` and `X-Forwarded-User` for Ollama's logs and for the usage ledger proposed in part 3.

Browser calls to Ollama's API (the prompt box on the Models page) carry the session cookie plus an `X-CSRF-Token` header, so another web site cannot make a signed-in user's browser call Ollama behind their back:

```
POST /api/chat   {"model":"gemma3:12b", ...}   as deniz.dev (allowlist llama3.2*, qwen3*, nomic-
embed-text*)
→ 403 {"error": "Not allowed to use model gemma3:12b."}
POST /api/chat   {"model":"qwen3:4b", ...}     as a Viewer
→ 403 {"error": "Missing permission model:run."}
```

Security measures

Threat	Measure
Password guessing	Lockout after 5 failures for 15 min; scrypt; equal timing for unknown users.
Stolen password	Optional TOTP; a used code cannot be replayed (`mfa_last_step`).
Signing up with someone else's address	No account until the e-mailed code is entered; the code goes only to that inbox.
Guessing a sign-up code	Codes are random, stored only as a hash, expire in 10 min; 5 wrong tries discard the sign-up; resends limited to one a minute and five in total.
Session fixation / hijack	Random 256-bit IDs, HttpOnly + SameSite cookies, ID rotated after MFA.

Threat	Measure
CSRF	Token in every form (separate tokens for the login and sign-up forms); browser API calls must send `X-CSRF-Token`.
Clickjacking, XSS	CSP, `frame-ancestors 'none'`, auto-escaping templates.

4.5 Verification and known limits

An automated suite of 19 pytest tests runs the gateway against an in-process stand-in for Ollama (`mock_ollama.py`, which reproduces the JSON shapes of the real API) and a fake mailer. It checks the RFC 6238 test vector and replay rejection, the policy matrix, login success and the generic failure message, login-form CSRF, lockout, the complete MFA flow including session rotation, unauthenticated API access, a Viewer blocked from running models, a Developer's allowlist and filtered model list, the OpenAI-compatible route, a disabled user losing access immediately, the CSRF header for browser calls, auditing of denials, and sign-up: a Bilkent subdomain becomes Developer, another domain becomes Viewer and can then sign in with its e-mail, wrong codes are limited, duplicates and weak passwords are rejected, and resends respect the cooldown. All 19 pass. We also ran the whole sign-up end to end against a real SMTP server (Figure 19); the screenshots in this report were taken from that running system with a scripted browser.

Limit	Path forward
Only browser access is covered; scripts and SDKs can't use the gateway	Add scoped, revocable API keys (`Authorization: Bearer …`) checked by the same PDP.
Single process, SQLite	Move sessions to Redis and the database to PostgreSQL; the gateway is then stateless and can run as several replicas.
Sign-up tells a visitor that an e-mail is already registered	Always show "check your e-mail" and send the existing owner a "you already have an account" message instead.
The code is sent while the visitor waits; a slow mail server slows the page	Hand messages to a background queue with retries.
Inline scripts require `'unsafe-inline'` in the CSP	Move scripts to static files and use nonces.

5 Conclusion

Ollama's architecture is a clear example of designing for one quality above all: making local inference effortless. A Go control plane with a single scheduler, out-of-process engines behind one interface, Docker-like model distribution and edge adapters for other APIs together deliver that. The same choices leave shared deployments under-served. Our two proposals target that gap from different sides and keep Ollama's priorities intact: the observability layer is an opt-in, event-based addition that cannot slow the inference path, and the IAM gateway adds authentication, fine-grained authorization, auditing and a better sign-in experience without changing a line of Ollama. They also reinforce each other: the gateway's per-user identity is what makes the extension's usage ledger meaningful.

References

1. Ollama source code, github.com/ollama/ollama, commit 42e911b (2 Oct 2026): `server/routes.go`, `server/sched.go`, `llm/server.go`, `llm/llama_server.go`, `mlxrunner/client.go`, `manifest/`, `envconfig/config.go`, `docs/faq.mdx`. MIT licence.
2. Ollama issue #3144 "add /metrics endpoint"; issue #9254 "Add support for opentelemetry tracing"; pull requests #16998 and #18508. github.com/ollama/ollama.

3. Cisco Talos / Cisco Security Blog, "Detecting Exposed LLM Servers: A Shodan Case Study on Ollama", September 2025.
4. Cybernews, report on CVE-2026-7482 (heap out-of-bounds read via crafted GGUF files, fixed in 0.17.1), May 2026.
5. P. Kruchten, "The 4+1 View Model of Architecture", IEEE Software 12(6), 1995.
6. L. Bass, P. Clements, R. Kazman, *Software Architecture in Practice*, 4th ed., Addison-Wesley, 2021 (quality attribute scenarios, patterns).
7. IETF RFC 6238 (TOTP), RFC 5321 (SMTP), RFC 3207 (STARTTLS); OASIS XACML 3.0 (PEP/PDP model).
8. Prometheus exposition format documentation; OpenTelemetry specification (OTLP).